

MARIA: Towards Efficient Approximate Maximum Inner Product Search on Large-Scale Vector Datasets

Xi Zhao¹, Yao Tian¹, Kai Huang², Zhonghan Chen¹, Bolong Zheng³, Ruiyuan Zhang¹,
Xiaofang Zhou¹

¹Hong Kong University of Science and Technology, Hong Kong, China

²Macau University of Science and Technology, Macau

³Huazhong University of Science and Technology, Wuhan, China

xzhaoca@cse.ust.hk, ytianbc@cse.ust.hk, kylehuangk@gmail.com, zchenhj@cse.ust.hk,

zblchris@gmail.com, zry@ust.hk, zxf@cse.ust.hk

ABSTRACT

As the cardinality and diversity of vector datasets increase, driven by the success of numerous embedding models for unstructured data, the approximate maximum inner product search (AMIPS) problem in high-dimensional vector spaces becomes increasingly important for various machine learning tasks. Existing AMIPS methods, such as locality-sensitive hashing (LSH) and graph-based methods, suffer from inadequate query accuracy, low query efficiency, and limited scalability for large-scale datasets.

To address these challenges, we propose a novel in-memory graph-based framework called MARIA for efficiently solving the AMIPS problem. Specifically, MARIA first partitions a dataset into different blocks based on norms of points and constructs an approximate proximity graph (APG) for each block to capture inner product relationships, thereby alleviating norm bias issues. It then connects all APGs by linking points across different APGs based on their angular similarity, facilitating fast AMIPS queries on inter-block APGs. We also introduce a lightweight LSH index to mitigate the significant cost of constructing the APG from scratch. To further enhance query efficiency and scalability, we develop a skip-list-like strategy to reduce the overall size of the APG, along with an LSH-based quantization technique to lower query costs. Additionally, we provide a theoretical analysis of query quality and computational complexity. Extensive experiments on real-world datasets demonstrate that MARIA outperforms existing methods in terms of query efficiency, accuracy, and scalability for large-scale datasets.

PVLDB Reference Format:

Xi Zhao¹, Yao Tian¹, Kai Huang², Zhonghan Chen¹, Bolong Zheng³, Ruiyuan Zhang¹, Xiaofang Zhou¹. MARIA: Towards Efficient Approximate Maximum Inner Product Search on Large-Scale Vector Datasets. PVLDB, 18(1): XXX-XXX, 2025.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/Jacyhust/MARIA>.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 18, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

1 INTRODUCTION

In recent years, there has been a significant surge in the availability of high-dimensional vector data (*i.e.*, vector representations of objects), primarily due to the proliferation of unstructured data and the remarkable success of deep neural network-based embedding models. These models have demonstrated their ability to embed various types of unstructured data, including videos [14], molecular structural information of drugs [10], and documents [30], into vectors. Subsequently, these vectors can be stored in a vector database to facilitate a wide range of applications, including retrieval-augmented generation (RAG) in large language models (LLMs), which employ vector search to mitigate LLM hallucinations. As a result, the *Maximum Inner Product Search (MIPS) problem*, which aims to find a vector (*i.e.*, a representation of an object) from a dataset D that has the maximum inner product with a given query q , plays a crucial role in these applications. Given a query point $q \in \mathbb{R}^d$ and a dataset $\mathcal{D} \subseteq \mathbb{R}^d$, the MIPS problem aims to find a point¹ x in $\mathcal{D}$ that has the maximum inner product $q^\top x$. A basic solution to the MIPS problem is the classic tree-based method [26, 28, 41], which works well in low-dimensional spaces. However, when the dimensionality increases to several hundred or higher, the search time increases exponentially [2, 8, 11] due to the “curse of dimensionality.” Therefore, a more efficient and sufficient alternative is to find the approximate MIPS result. In this paper, we study the problem of *Approximate Maximum Inner Product Search (AMIPS)* (see Definition 2), which identifies the top- k “approximately” most relevant objects for query q over the collection $\mathcal{D}$.

Existing solutions for the AMIPS problem mainly consist of locality-sensitive hashing (LSH)-based methods [3, 21, 22] and graph-based methods [20, 23, 34]. LSH-based methods first reduce the AMIPS problem to an approximate nearest neighbor search (ANNS) problem [4, 29, 31, 48, 59–61], and further apply LSH indexes for addressing ANNS problem. In contrast, graph-based methods [20, 34, 58] builds an approximate proximity graph (APG) to facilitate AMIPS. In the APG, each vertex represents a data point in the dataset and each edge connects two vertices with a smaller distance. Based on APG, AMIPS can be solved by adopting beam search in the APG [20, 34, 58]. However, existing solutions suffer from the following three challenges.

(C1) *Inadequate query accuracy.* Graph based methods are well known for their high query accuracy in the approximate nearest neighbor search (ANNS) problem. This effectiveness stems from

¹In this paper, we use the terms “points” and “vectors” interchangeably.

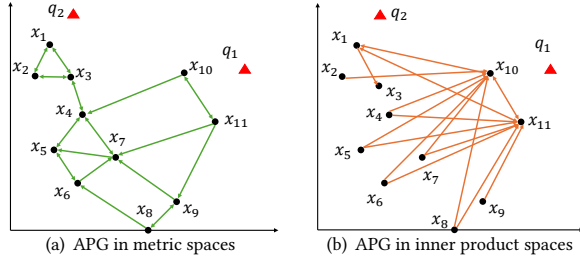


Figure 1: APG in metric spaces and inner product spaces.

the triangle inequality in metric spaces, which ensures that (i) *the neighbor of a neighbor is very likely to be a neighbor*, and (ii) *the nearest neighbors of a query point are in close proximity and can be reached from one another in the approximate graph with only a few hops*. Consider the APG shown in Figure 1(a), which includes points x_1 , x_2 and x_3 in a metric space. Here, x_1 is a neighbor of x_2 , and x_2 is the neighbor of x_3 . Consequently, x_1 is also a neighbor of x_3 . For the query point q_2 , its two neighbors, x_1 and x_3 , are located within a small region of the metric space.

However, when applied to the AMIPS problem, performance deteriorates because the aforementioned properties do not hold in inner product spaces, where the inner product is non-metric. As a result, the common search strategy in APGs used for ANNS becomes inefficient for AMIPS. Figure 1(b) illustrates the APG in an inner product space. Observe that x_9 is a neighbor of x_{11} , and x_{11} is a neighbor of x_1 , but x_9 is not a MIPS neighbor of x_1 due to their large angular distance. Additionally, the AMIPS problem suffers from *norm bias* issues, *i.e.*, points with large norms are more likely to be MIPS neighbors of other points. Consequently, points with large norms tend to be mutually connected, making it difficult for a point with modest norms to be reached even if they are promising MIPS neighbors of the query point. For example, in Figure 1(b), x_1 , x_{10} and x_{11} are mutually connected and do not regard other points as a MIPS neighbor. Once the search enters into x_1 , x_{10} and x_{11} , we cannot find other points. Compared to graph based methods, LSH based methods achieve a worse query accuracy due to the *hash-boundary issue* [21, 49, 58]: some points that are close in the original space may be mapped into different hash buckets, thus not being identified by LSH-based approaches as neighbors [58].

(C2) *Low query efficiency*. For graph-based methods, unlike in metric spaces, the MIPS results of a query point q are often widely distributed in the inner product space, while two inner product neighbors of q may have a small inner product with each other. This leads to the APG search encountering multiple disjoint local maxima, making efficient convergence difficult. As shown in Figure 1(b), although q_1 identifies x_1 and x_9 as two of its MIPS neighbors, these points are positioned on opposite sides of q_1 and have an extremely small inner product with each other. LSH-based methods address the AMIPS problem by accessing the hash buckets near q across multiple hash tables. However, due to their poor estimation quality, they require significantly more time to achieve the same level of accuracy as graph-based methods.

(C3) *Limited scalability for large-scale dataset*. The limited scalability of LSH-based and graph-based methods for large-scale datasets

arises from different challenges. For LSH-based methods, the number of points in a hash bucket increases with data cardinality, leading to a larger candidate size during query processing. In the worst cases, the query cost of some LSH-based methods [24, 59] can approach $O(n)$, where n is the cardinality of the datasets, making them impractical for large-scale datasets. On the other hand, graph-based methods face significant construction costs, as they require identifying appropriate neighbors for all points to build the APG. Additionally, the time needed to find the neighbors of a point also grows with data cardinality.

In this paper, we present a novel in-memory graph-based framework called MARIA (approximate Maximum inner Product search In vector datasets) for efficiently answering the AMIPS problem on large-scale vector datasets. Specifically, MARIA first partitions a dataset into different blocks based on norms of points and constructs an APG for each block to capture inner product relationships, thereby alleviating norm bias issues. Searching within a block using the APG is much more efficient and accurate than searching the entire dataset, as the points within a block ensure that a MIPS neighbor of the query’s MIPS neighbor is likely to be the query’s MIPS neighbor when the points in a block have similar norms. Then, MARIA connects all APGs by linking points across different APGs based on their angular similarity, facilitating fast AMIPS queries on inter-block APGs.

In addition, we develop three optimization techniques to further enhance the query performance and scalability of MARIA. First, we design a lightweight yet efficient LSH index to mitigate the significant cost of constructing the novel APG from scratch. For each point, we retrieve angular neighbors from the LSH index, which serve as entry points for refining the APG. These entry points streamline the query process for identifying high-quality neighbors for each point, thereby reducing construction costs and improving support for large-scale datasets. Second, we implement a skip-list-like strategy to selectively choose inter-block edges, thereby reducing the overall size of the APG. This approach significantly decreases the construction cost and index size of inter-block edges while still ensuring query accuracy, improving support for large-scale datasets. Finally, to further boost query efficiency, we propose a compact yet accurate LSH-based quantization technique to quickly estimate the inner product of two points, allowing us to filter out unqualified points during the search. This significantly reduces the number of points accessed during the graph search.

The major contributions of this paper are as follows.

- We provide a comprehensive analysis of state-of-the-art LSH-based and graph-based methods for AMIPS, and identify the challenges that highlight the inefficiencies of existing approaches.
- We propose a novel graph-based framework called MARIA with an innovative graph structure to efficiently address the AMIPS problem. MARIA not only alleviates the norm bias issue in the MIPS problem but also captures angular similarity to facilitate fast AMIPS queries.
- We present three optimizations to further enhance the query efficiency and scalability of our MARIA framework. In particular, we design a lightweight yet efficient LSH index to mitigate the cost of constructing an APG and develop a skip-list-like strategy to reduce the overall size of the APG, along with an LSH-based

quantization technique to lower query costs. Additionally, we provide a theoretical analysis of query quality and computational complexity.

- Extensive experiments on real-world datasets demonstrate that MARIA outperforms existing methods in terms of query efficiency, accuracy, and scalability for large-scale datasets.

The rest of the paper is organized as follows. Section 2 reviews related work. Section 3 presents the preliminaries. Section 4 introduces the MARIA framework, while Section 5 discusses the optimization techniques. Section 6 provides a theoretical analysis. Section 7 presents experimental studies that offer insights into the performance of the proposed MARIA and its main competitors. Finally, we conclude the paper in Section 8.

2 RELATED WORK

The approximate Maximum Inner Product Search (AMIPS) problem was developed as a branch of approximate nearest neighbor search (ANNS) in the database community. Therefore, like ANNS problem, solutions to the AMIPS problem predominantly utilize methods based on locality-sensitive hashing (LSH) [25] and graph-based methods [36, 45] by replacing $\|q, x\|$ with $-q^\top x$ in the algorithms. But as inner product is non-metric, these methods have limited efficiency due to the mentioned challenges earlier, so there are some strategies proposed for transforming AMIPS into ANNS [6, 24, 43, 44, 55, 59], which can then be solved via well-established methods for ANNS. In addition, some methods are specially designed for AMIPS by considering the property of AMIPS [32, 46, 47, 62].

2.1 Take AMIPS as a kind of ANNS

Since ANNS and AMIPS are similar problems and ANNS problem has been well studied via LSH and graph based methods [25, 36, 45], AMIPS can be taken as a kind of ANNS by replacing $\|q, x\|$ with $-q^\top x$ simply. RMMH [25] is an LSH based methods designed for AMIPS. By training an LSH function family that are locality-sensitive to the inner product, it can extend the classic LSH framework for ANNS into AMIPS. However, the learned hash functions are non-sensitive to the norms of the points and suffer from the norm bias issue. HNSW [34], one of the most effective in-memory graph-based methods, approximates the Delaunay Graph [27] to efficiently solve ANNS. The Delaunay graph [27] is known to solve the NNS problem exactly. Therefore, existing graph-based methods aim to better approximate the Delaunay graph in the inner product space. Ip-NSW [34] addresses the AMIPS problem by replacing the similarity function in HNSW from *L2-distance* to *inner product*, demonstrating that a Delaunay graph in an inner product space can also ensure exact MIPS results. Such a strategy that replaces the similarity function in HNSW from *L2-distance* to *inner product* is also used in DiskANN [45] by Microsoft and Milvus due to its simplicity. However, as inner product is non-metric, the greedy search in the APGs for AMIPS can not expect to have as high performance as that for ANNS due to the analyzed challenges in the earlier section.

2.2 From AMIPS to ANNS

Other direction to solve AMIPS problem is to transform AMIPS into ANNS and then adopt well-established ANNS methods to solve

it. L2-ALSH [44] and Sign-ALSH [43] employ L2 transformation and correlation transformation to convert the AMIPS problem into an ANNS problem, subsequently applying LSH-based methods to solve it [6, 43, 44]. This strategy has led to the development of several LSH-based methods with asymmetric transformations, such as Simple-LSH [37], Range-LSH [55], H2-ALSH [24], and FARGO [59], which are favored for their straightforward structure and superior performance. However, it does not mean that the transformation comes without drawbacks. The straight effect of these transformations is the transformation distortion error [24], which indicates even though $q^\top x_1$ and $q^\top x_2$ are very different and easy to tell them apart but their distances to q may be almost the same and become hard to distinguish them after the transformation. In this case, it will be much more difficult to answer the transformed ANNS problem efficiently. In addition, [55] analyzed that these asymmetric transformations are not suitable for graph based methods to AMIPS since in the transformed space, a point prefer to connect to the points with smaller norms, which is counterproductive to MIPS.

2.3 The specialized solutions for AMIPS

Except for those algorithms that solve AMIPS by considering the existing solutions to ANNS, some studies aim to directly design specialized graph based indexes for AMIPS. Ip-NSW⁺ highlights the limitations of ip-NSW, stemming from the differences between inner product and L2-distance, noting that a MIPS neighbor of a MIPS neighbor is unlikely to be a MIPS neighbor again (*i.e.*, *challenge 2*). Consequently, the search in ip-NSW involves numerous unnecessary computations, as many edges in ip-NSW are unsuitable for AMIPS. Ip-NSW⁺ enhances ip-NSW's performance by using an additional APG in angular space to generate entry points for continuous AMIPS searches. However, optimizing only the entry points offers limited improvements due to the large number of unsuitable edges in ip-NSW. Other works, such as Möbius-Graph [62], IPDG [47], and NAPG [46], focus on designing better strategies to approximate the Delaunay graph in an inner product space. These methods achieves limited performance since their improvements help little for addressing the three challenges in AMIPS problem. Additionally, other methods, such as learning, sampling, and quantization based methods, have been proposed for AMIPS [15, 19, 42]. Shen et al. [42] propose a binary code learning framework that preserves the inner product among raw data vectors. BanditMIPS [50] estimates the inner product for each point by subsampling coordinates and adaptively evaluates more coordinates for more promising points. However, these methods focus on reducing the cost of computing an inner product but have high query complexity to achieve high query accuracy, limiting their scalability to large-scale datasets.

2.4 Disk based methods in Vector Databases

In most modern databases that support vector search, such as Milvus [35], Pinecone [39], and Azure [5], AMIPS (Approximate Maximum Inner Product Search) is considered as important as ANNS (Approximate Nearest Neighbor Search). However, nearly all of these databases treat the AMIPS problem as a variant of ANNS and address it using methods like HNSW [34] or DiskANN [45]. DiskANN, a disk-based approach, introduces a novel graph index called Vamana to reduce I/O costs. However, when comparing HNSW and

Vamana in memory, their performance is nearly identical [51]. As a result, ip-NSW remains the superior choice for addressing AMIPS in memory.

3 PRELIMINARIES

Let $\mathbb{R}^d$ be a d -dimensional vector space, $x_1^\top x_2$ the inner product between points $x_1, x_2 \in \mathbb{R}^d$, and $\langle x_1, x_2 \rangle$ the angle between x_1 and x_2 . In this paper, we focus on the AMIPS problem in memory. Frequently used notations are summarized in Table 1.

3.1 Problem Definition

DEFINITION 1 (MAXIMUM INNER PRODUCT SEARCH). Given a dataset $\mathcal{D} \subseteq \mathbb{R}^d$ with $|\mathcal{D}| = n$ and a query point $q \in \mathbb{R}^d$, the maximum inner product search (MIPS) returns a point $x^* \in \mathcal{D}$:

$$x^* = \arg \max_{x \in \mathcal{D}} q^\top x. \quad (1)$$

DEFINITION 2 ((c, k)-APPROXIMATE MAXIMUM INNER PRODUCT SEARCH [57]). Given a dataset $\mathcal{D}$, a query point $q \in \mathbb{R}^d$, an approximation ratio $0 < c < 1$, and a positive integer k , a (c, k)-approximate maximum inner product search (AMIPS) returns k points $x_1, \dots, x_k \in \mathcal{D}$ sorted in descending order w.r.t. their inner products with q . For $\forall x_i, i = 1, \dots, k$, we have $q^\top x_i \geq c \cdot q^\top x_i^*$, where $q^\top x_i^*$ is the i -th maximum inner product.

When all points in $\mathcal{D}$ have the identical norm, the AMIPS problem is equivalent to the approximate nearest neighbor search (ANNS) problem under the angular distance.

DEFINITION 3 ((c', k)-APPROXIMATE NEAREST NEIGHBOR SEARCH). Given a dataset $\mathcal{D}$, a query point $q \in \mathbb{R}^d$, an approximation ratio $c' > 1$, and a positive integer k , a (c', k)-approximate nearest neighbor search (ANNS) returns k points $x_1, \dots, x_k \in \mathcal{D}$ sorted in ascending order w.r.t. their angular distance to q . For $\forall x_i, i = 1, \dots, k$, we have $\langle q, x_i \rangle \leq c' \cdot \langle q, x_i^* \rangle$, where x_i^* is the i -th nearest neighbor of q in $\mathcal{D}$.

Based on the definition above, it is clear that

$$\arg \min_{x \in \mathcal{D}} \langle q, x \rangle = \arg \max_{x \in \mathcal{D}} \frac{q^\top x}{\|q\| \|x\|} = \arg \max_{x \in \mathcal{D}} q^\top x, \quad (2)$$

where the first equality holds because of $\cos \langle q, x \rangle = \frac{q^\top x}{\|q\| \|x\|}$, and the second equality follows from the identical norm in $\mathcal{D}$. Therefore, solving the MIPS problem is equivalent to finding a point x with the smallest angular distance to q .

REMARK 1. For clarity, we simplify (c, k)-AMIPS and (c', k)-ANNS problems to c -AMIPS and c' -ANNS when $k = 1$.

3.2 Locality Sensitive Hashing based Methods

Locality-sensitive hashing (LSH) is one of the most popular tools for solving the ANNS problem due to its simple structure, sublinear query cost, and rigorous quality guarantee. The formal definition of LSH is given below.

DEFINITION 4 (LOCALITY-SENSITIVE HASHING). Given a distance function dist , a distance threshold r , an approximation ratio $c > 1$, and two probability values p_1, p_2 , where $p_1 > p_2$, a family $\mathcal{H} = \{h : \mathbb{R}^d \rightarrow U\}$ is called (r, cr, p_1, p_2)-locality sensitive, if for any $x_1, x_2 \in \mathbb{R}^d$, it satisfies the following two conditions:

Table 1: List of Key Notations

Notation	Definition
$\mathcal{D} \subseteq \mathbb{R}^d$	A dataset
$n = \mathcal{D} $	The dataset cardinality
x, u, v	A point (a.k.a. vector) in $\mathcal{D}$
$N(x)$	x 's neighbor set
q	A query point
$q^\top x$	The inner product between q and x
$\langle q, x \rangle$	The angle between q and x
s	The number of blocks
M	The maximum inner-block degree
M'	The maximum inter-block degree between two blocks

- (1) If $\text{dist}(x_1, x_2) \leq r$, then $\Pr[h(x_1) = h(x_2)] \geq p_1$;
- (2) If $\text{dist}(x_1, x_2) \geq cr$, then $\Pr[h(x_1) = h(x_2)] \leq p_2$.

A typical LSH for angular distance is defined as follows [9]:

$$h(x) = \text{sign}(a^\top x), \quad (3)$$

where a is a d -dimensional random vector with each entry independently chosen from a 2-stable distribution, i.e., the standard normal distribution $N(0, 1)$.

3.3 Graph based Methods

The graph-based methods build the approximate proximity graphs (APG) for vector similarity search.

DEFINITION 5. Given a similarity function $f : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ and a dataset $\mathcal{D} \in \mathbb{R}^d$, an approximate proximity graph (APG) $\mathcal{G} = (V, E)$ is a graph where the vertex sets $V = \mathcal{D}$ and the edge sets $E \subseteq \{(x_1, x_2) | x_1, x_2 \in \mathcal{D}\}$. The edge $(x_1, x_2) \in E$ implies that x_2 has a high similarity to x_1 in $\mathcal{D}$ according to the function f .

REMARK 2. In a metric space $\mathcal{M} = (\mathbb{R}^d, \text{dist})$, we define the similarity function f as $f(x_1, x_2) = -\text{dist}(x_1, x_2)$.

Terminology. Given an APG $\mathcal{G} = (V, E)$, for an edge $(x_1, x_2) \in E$, we denote x_2 as the *neighbor* of x_1 . (x_1, x_2) is an *out-edge* of x_1 and an *in-edge* of x_2 . The set $N(x) = \{v | (x, v) \in E\}$ is the *neighbor set* of x in $\mathcal{G}$, while $RN(x) = \{u | (u, x) \in E\}$ denotes the *reverse neighbor set* of x in $\mathcal{G}$. We define $|N(x)|$ as the *out-degree* of x and $|RN(x)|$ as the *in-degree* of x . For clarity, we often refer to the out-degree of x simply as the *degree* of x .

REMARK 3. Graph-based methods typically do not explicitly utilize the approximation ratio c to control the quality of the query. To avoid confusion, we abbreviate (c, k)-AMIPS as k -AMIPS for simplicity.

3.4 Limitations of Existing AMIPS methods

As graph based methods have been demonstrated to be the most promising methods for ANNS, we analyze why they are inefficient when answering AMIPS directly or a transformed ANNS problem.

This effectiveness of graph based methods for ANNS stems from the triangle inequality in metric spaces, which ensures that (i) the neighbor of a neighbor is very likely to be a neighbor, and (ii) the nearest neighbors of a query point are in close proximity and can be reached from one another in the approximate graph with only a few

hops. However, when applied to the AMIPS problem, performance deteriorates because these properties do not hold in inner product spaces, where the inner product is non-metric. As a result, the common search strategy in APGs used for ANNS becomes inefficient for AMIPS. In addition, unlike in metric spaces, the MIPS results of a query point q are often widely distributed in the inner product space, while two inner product neighbors of q may have a small inner product with each other. This leads to the APG search encountering multiple disjoint local maxima, making efficient convergence difficult. Furthermore, graph-based methods face significant construction costs, as they require identifying appropriate neighbors for all points to build the APG.

As for answering a transformed ANNS problem via graph based methods by transforming AMIPS into ANNS, it will be inefficient since the transformed dataset has a skewed distribution. As analyzed in ip-NSW [55], in the transformed space, a point prefers to connect to the points with smaller norms because these points are more likely to have a smaller distance to each other. However, the norm bias issue tells us that AMIPS problem prefers the points with larger norms. Therefore, it becomes inefficient to answer ANNS in the transformed datasets by adopting APGs.

4 THE MARIA FRAMEWORK

In this section, we introduce a novel graph-based framework called MARIA, designed to efficiently solve the AMIPS problem. An overview of MARIA is depicted in Figure 2. MARIA addresses the aforementioned challenges (*i.e.*, inadequate query accuracy, low query efficiency, and limited scalability) as follows.

Given a dataset $\mathcal{D}$, MARIA first adopts a *norm-based Partition* strategy (see Section 4.1) to partition $\mathcal{D}$ into blocks $\mathcal{D}_1, \mathcal{D}_2, \dots, \mathcal{D}_s$ based on norms of points. For each block, MARIA constructs an APG (*i.e.*, inner-block APG) (see Section 4.2) to connect inner-block points to take advantage of the state-of-the-art performance of graph-based methods for ANNS. To support queries over all blocks, MARIA builds inter-block edges between two blocks $\mathcal{D}_i$ and $\mathcal{D}_j$ to linking points $u \in \mathcal{D}_i$ and $v \in \mathcal{D}_j$ based on the angular similarity of u and v . Based on the novel APG, MARIA can efficiently process the AMIPS query (see Section 4.3). Moreover, MARIA integrates three optimizations to further enhance query efficiency and scalability (see Section 5). Compared to existing solutions, MARIA has the following advantages:

- (i) *High-quality query results.* MARIA constructs an APG within each block, where points have similar norms, effectively mitigating norm bias issues. Searching within a block using the APG is more accurate than searching the entire dataset.
- (ii) *Fast query processing.* MARIA enables efficient searches by enhancing neighbor relationships within each APG. For multiple searches across all blocks, entry points generated in the LSH framework, along with query results from one block, guide searches in subsequent blocks, facilitating faster convergence. Additionally, a quantization-based pruning condition (see Section 5.1) is employed to further reduce computational costs.
- (iii) *Low construction cost.* By generating neighbor sets for each point through searches within its block, MARIA ensures low construction costs using efficient query strategies. Additionally, a skip-list-like mechanism (see Section 5.2) reduces the number of searches required during construction, further minimizing costs.

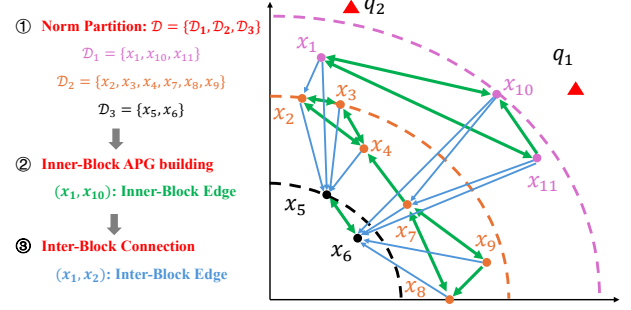


Figure 2: The MARIA framework

- (iv) *Desirable scalability.* Leveraging its novel graph structure and effective optimizations, MARIA achieves superior indexing efficiency and query performance, making it well-suited for large-scale datasets.

EXAMPLE 1. Consider $\mathcal{D}$ in Figure 2. With $\alpha = 0.7$, $\mathcal{D}$ is divided into three blocks: $\mathcal{D}_1 = \{x_1, x_{10}, x_{11}\}$, $\mathcal{D}_2 = \{x_2, x_3, x_4, x_7, x_8, x_9\}$, and $\mathcal{D}_3 = \{x_5, x_6\}$ via norm-based partition. An APG is constructed within each block, as shown by the green lines. Unlike the APG in Figure 1(b), where all edges are captured by x_1, x_{10} , and x_{11} , MARIA allows each point to be probed, mitigating norm bias issues. Inter-block edges, depicted by the blue lines (e.g., (x_1, x_2)), connect different blocks. To perform a 2AMIPS query for q_2 , MARIA starts with block $\mathcal{D}_1$ and finds x_1 . It then proceeds to search in $\mathcal{D}_2$ starting from x_2 , as x_2 is the inter-block neighbor of x_1 , significantly reducing the search cost in $\mathcal{D}_2$. After searching these blocks, x_1 and x_2 are returned as the 2AMIPS results.

4.1 Norm-based Partition

AMIPS results are influenced by both vector norms and their angles relative to the query, leading to a wide distribution of results that makes constructing a single APG challenging. To handle this, MARIA introduces a norm-based partitioning strategy, which divides the dataset into multiple blocks based on the norms of vectors. Vectors within each block have similar norms. This approach allows MARIA to treat the AMIPS problem within each block as equivalent to the ANNS problem in the angular space. As shown in Figure 2, all data points are partitioned into three blocks $\mathcal{D}_1, \mathcal{D}_2$ and $\mathcal{D}_3$ illustrated with different colors. Let $\mathcal{D}_1, \mathcal{D}_2, \dots, \mathcal{D}_s$ denote partitioned blocks, where the points in $\mathcal{D}_1$ have the largest norms. We adopt a simple but effective partition strategy that focuses on controlling the norm range in a block. Specifically, given a ratio $\alpha \in (0, 1)$, for each block $\mathcal{D}_i$, let x_1, x_2 be any two points in $\mathcal{D}_i$, it satisfies $\alpha \cdot \|x_1\| < \|x_2\|$. This method effectively divides the space into onion-like concentric hyperspheres, with each block representing a hyperring of vectors.

LEMMA 1. Given a norm ratio α , a correct angular neighbor of the query point q in a block $\mathcal{D}_i$ is the α -AMIPS neighbor of q .

The proof of this lemma is provided in Section 6.1. We have demonstrated the AMIPS problem becomes equivalent to the ANNS problem when all points have the identical norm. Lemma 1 reveals that the equivalence still exists in a block. Therefore, the AMIPS problem can be solved more accurately via the APG in a block than

in the whole dataset since the block satisfies the assumption that the neighbor of a neighbor is very likely to be a neighbor.

4.2 APG construction in MARIA

After partitioning $\mathcal{D}$ into blocks $\mathcal{D}_1, \mathcal{D}_2, \dots, \mathcal{D}_s$, MARIA construct an APG for each block and then connects any two of blocks $\mathcal{D}_i$ and $\mathcal{D}_j$ by linking points $u \in \mathcal{D}_i$ and $v \in \mathcal{D}_j$ based on the angular similarity of u and v .

4.2.1 Inner-Block Construction. The APG in the block $\mathcal{D}_i$, denoted as $\mathcal{G}_i = (V_i, E_i)$, is a directed MIPS graph. Here, $E_i = \{(u, v) | u, v \in \mathcal{D}_i\}$ is the edge set where v is an AMIPS neighbor of u in $\mathcal{D}_i$. Building the APG $\mathcal{G}_i$ is extremely time-consuming since we are required to find MIPS neighbors for each point. There are mainly two commonly used strategies that are designed to lower the construction cost. In the first strategy: NN-Descent[18, 23, 38], the APG is improved from an initial APG iteratively. In the initial APG, each vertex is attached with M neighbors randomly. At one epoch, for any two neighbors u and v of a point x , we compute $u^\top v$ and updates the neighbor sets of u and v , refining the neighbor quality of u and v . The second strategy builds an APG via consecutive insertion strategy where the point is inserted into the APG one by one, as that in ip-NSW [36] and LSH-APG [58]. When inserting a point x , we conduct a MIPS search for x to find its neighbors in the current APG to generate its edge sets. The cost of the consecutive insertion strategy depends on the efficiency of the MIPS search in the APG. Although NN-Descent is demonstrated to build a higher-quality APG [13], the built APG via NN-Descent achieves a limited query performance due to the low connectivity [20] and unified degree M [58]. Low connectivity makes many points unreachable from other vertex and the unified degree fails to consider the varied distribution in the real-world dataset. Therefore, we choose the consecutive insertion strategy to build a varied-degree APG in $\mathcal{D}_i$ for a better query performance.

The construction is described in Algorithm 1. We build $\mathcal{I}_G$ via the consecutive insertion strategy as described in Algorithm 1. To insert a point x into $\mathcal{G}_i$, we find its M_0 AMIPS results in $\mathcal{D}_i$ as its neighbor set $N(x)$ (Line 4). To find the M_0 AMIPS results, we conduct a beam search for x in the current APG of $\mathcal{D}_i$, as described in Algorithm 3. The details of search is explained in Section 4.3. After finding $N(x)$, we insert these edges into $\mathcal{G}_i$ (Line 6) and update the neighbor set for each point v in $N(x)$ with x . Each point v stores at most M AMIPS results in its neighbor set (Line 8-10). In the final APG, the degree of each point varies in a range $[M_0, M]$. All the points are allocated with M_0 neighbors once inserted. Then, once it is selected as the AMIPS neighbor of another points, it can append a neighbor. Therefore, the points that are liable to be the MIPS result of other points are likely to have more neighbors. Such a varied-degree strategy fully considers the difference among the data distribution and facilitates the efficient search. The other parameter efC is adopted for controlling the query quality where a larger efC achieves a higher quality but incurs a larger query cost. And M controls the maximum degree because a too large degree will increase the query cost in turn [52, 58].

4.2.2 Inter-Block Connection. For clarity, we define two types of edges in the graph. The first type, *inner-block edges*, represents the edges built within each block, as described in Section 4.2.1. These

Algorithm 1: Building APG in a block

Input: A block $\mathcal{D}_i$, parameters efC , M_0 and M
Output: An APG in $\mathcal{D}_i$

```

1  $E_i \leftarrow \emptyset$ ;
2  $N_I \leftarrow$  random  $M_0$  inserted points in  $\mathcal{D}_i$ ;
3 for each point  $x \in \mathcal{D}_i$  do
4    $N_f \leftarrow$  Call AMIPS( $x, \mathcal{G}_i, N_I, N_I, efC$ );
5   for each  $v \in N_f$  do
6      $E_i \leftarrow E_i \cup \{(x, v)\}$ ;
7      $E_i \leftarrow E_i \cup \{(v, x)\}$ ;
8     if  $|N(v)| > M$  then
9        $u^* \leftarrow \arg \min_{u \in N(v)} u^\top v$ ;
10       $E_i \leftarrow E_i - \{(v, u^*)\}$ ;
11  $\mathcal{G}_i \leftarrow \{\mathcal{D}_i, E_i\}$ ;
12 return  $\mathcal{G}_i$ ;
```

Algorithm 2: Inter-Block Connection

Input: APGs $\{\mathcal{G}_i = (\mathcal{D}_i, E_i)\}_{i=1:s}$ and parameters M', efC
Output: An APG $\mathcal{I}_G$

```

1  $\mathcal{I}_G \leftarrow \cup_{i=1}^s \mathcal{G}_i$ ;
2 for  $i \leftarrow 1$  to  $s$  do
3   for  $j \leftarrow i + 1$  to  $s$  do
4     for each  $u \in V_i$  do
5        $N_I \leftarrow$  random  $M'$  inserted points in  $\mathcal{D}_j$ ;
6        $N_f \leftarrow$  Call AMIPS( $u, \mathcal{G}_j, N_I, M', M'$ );
7       for each  $v \in N_f$  do
8          $\mathcal{I}_G \leftarrow \mathcal{I}_G \cup \{(u, v)\}$ ;
9 return  $\mathcal{I}_G$ ;
```

edges capture the inner product relationships between points in the same block, alleviating norm bias issues due to their similar norms. The second type, *inter-block edges*, connects data points across blocks. Let $\mathcal{D}_1$ represent the partition with the largest norms, we demonstrate the procedure of finding inter-block edges in Algorithm 2. Specifically, for each two blocks $\mathcal{D}_i$ and $\mathcal{D}_j$ where $i < j$ and any a point $u \in \mathcal{D}_i$, we find M' angular neighbors for u in $\mathcal{D}_j$. The inter-block edges are unidirectional and they are always incident from the point with a larger norm to the point with a smaller norm. Since the search always begins from $\mathcal{D}_1$ and an inter-block edge (u, v) serves as the guidance for quickly determining the high-quality candidate v for the search in v 's block once we find the MIPS neighbor u in u 's block.

4.3 AMIPS Query Processing

The AMIPS search in MARIA conducts a series of search block by block. As shown in Algorithm 3, the AMIPS query in a block starts from an initial entry point set EPs and generate the AMIPS result for the query point q via a cyclic probing in the APG. In each epoch, the point e_p that has the maximum inner product to q in EPs is selected as the entry point (Line 2). For each neighbor x of e_p in the APG, we compute its inner product to q and update it into sets EP and R . The result set R stores the best ef points found so far. The search procedure repeats until we cannot find a point in EPs that

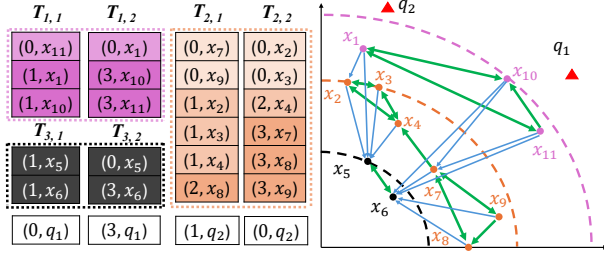
Algorithm 3: AMIPS($q, \mathcal{G}, EPs, R, k, ef$)

Input: A query point q , the APG $\mathcal{G}$, the initial entry point EPs , the result sets R , the parameters k and ef

Output: k points in $\mathcal{G}_i$

```

1 while  $|EPs| > 0$  do
2    $e_p \leftarrow \arg \max_{u \in EP} q^\top u$ ;
3   Remove  $e_p$  from  $EP$ ;
4    $x_f \leftarrow \arg \min_{u \in R} q^\top u$ ;
5   if  $|R| \geq ef$  and  $q^\top e_p < q^\top x_f$  then
6     break;
7   for  $x \in N(e_p)$  do
8     if  $x$  has not been visited then
9       Update  $x$  into  $EPs$  and  $R$ ;
10    while  $|R| > ef$  do
11       $x' \leftarrow \arg \min_{u \in R} q^\top u$ ;
12      Remove  $x'$  in  $R$ ;
13  $R \leftarrow$  The best  $k$  AMIPS results to  $q$  in  $R$ ;
14 return  $R$ ;
```

**Figure 3: LSH-based Initialization**

has a larger inner product to q than any of the best ef points in R (Line 5). Then, the search terminates and we return k best points in R as the AMIPS results of q . The parameter ef is set to balance the query efficiency and accuracy. A higher ef makes the termination harder (Line 5), which allows us to access more points to refine the query results but incurs a higher query cost.

To enhance the query efficiency, the search among different blocks are not independent. As described in Algorithm 4, the search begins from the block $\mathcal{D}_1$ since the points here have the largest norms. Then, we adopt the more accurate initial set EPs and result set R for the search in the next blocks to guide the search better (Line 7). For the search in $\mathcal{D}_{i+1}$, the initial result set R consists of the best k points found in the former i blocks, which enables the search terminate faster if there are few qualified results in $\mathcal{D}_{i+1}$. Moreover, we refine the initial entry point set EPs by accessing the inter-block neighbors of the points in R . For a point u in R , u 's neighbors in $\mathcal{D}_{i+1}$ will be chosen as the entry points. Generating higher-quality entry points greatly reduces the hop numbers required to terminate the algorithm, and thus improve the query efficiency. Moreover, a closer entry point decreases the probability that the query terminates at a local maximal x where $q^\top x$ is still very small.

5 OPTIMIZATIONS

In this section, we introduce several optimization techniques to further enhance MARIA's performance, including (a) an LSH-based

Algorithm 4: k ANN Query($q, \{\mathcal{G}_i\}, k, ef$)

Input: A query q , the APGs $\{\mathcal{G}_i\}$ and parameters k and ef

Output: k points in $\mathcal{I}_G$

```

1  $N_I \leftarrow$  random  $M_0$  points in  $\mathcal{D}_1$ ;
2  $R \leftarrow N_I$ ;
3 for  $i \leftarrow 1$  to  $s$  do
4   if  $|R| \geq ef$  and  $\|q\| \cdot \max_{x \in \mathcal{D}_i} \|x\| < \min_{x \in R} q^\top x$  then
5     break;
6   end
7    $R \leftarrow$  Call AMIPS( $q, \mathcal{G}_i, N_I, R, ef$ );
8    $N_I \leftarrow \{v \in \mathcal{D}_{i+1} | u \in R, (u, v) \in \mathcal{I}_G\}$ ;
9 end
10 return  $R$ ;
```

Algorithm 5: The MARIA framework

```

1  $\mathcal{D}_1, \mathcal{D}_2, \dots, \mathcal{D}_s \leftarrow$  Partition  $\mathcal{D}$  to blocks;
2  $\mathcal{I}_H \leftarrow$  Call LSHInit;
3 for each  $\mathcal{D}_i$  do
4   Generate initial neighbor sets in  $\mathcal{I}_{H_i}$  for each point;
5    $\mathcal{I}_{G_i} \leftarrow$  Call BuildingAPG;
6  $\mathcal{I}_H \leftarrow$  the union of all  $\mathcal{I}_{H_i}$ ;
7  $\mathcal{I}_G \leftarrow$  Call Inter-Block Connection;
8 return  $\mathcal{I}_H$  and  $\mathcal{I}_G$ ;
```

initialization; (b) a skip-list-like mechanism; and (c) a quantization and pruning technique.

- **LSH-based Initialization:** In section 4.2, the APG for each block was constructed using random initialization for search in the APG, which could result in high cost to converge the search, enlarging the construction costs. To address this, we introduce a lightweight LSH-based framework to efficiently select initial neighbors for each vertex, reducing the cost of search in an APG for generating the neighbor sets.
- **Skip-List-like Mechanism:** In Section 4.2.2, we introduce a naive inter-block connection mechanism, which adds notable complexity and inefficiency. To optimize this, we propose a novel skip-list-like strategy that reduces the number of inter-block edges and redistributes more inter-block edges to the blocks that are close to each other, significantly mitigating the index size and construction cost without scarifying the query performance.
- **Quantization and Pruning:** To further boost search efficiency, MARIA incorporates a quantization technique that exploits the properties of LSH to estimate inner products between points. This allows for the efficient pruning of unqualified candidates during the search, reducing the number of distance computations needed and significantly accelerating the query processing.

The complete MARIA framework, incorporating all optimizations, is outlined in Algorithm 5.

5.1 LSH-based Initialization

5.1.1 Building LSH framework. $\mathcal{I}_H$ consists of L hash tables for each block $\mathcal{D}_i$. In each hash table T_i , we randomly select K hash functions, $h_{i,1}, \dots, h_{i,K}$ from the hash family $\mathcal{H}$, as defined in Equation 3. Then, for a point x , we compute its K hash values.

As $h_{l,j}(x) \in \{0, 1\}$, we can transform the K -bit hash values of x , $h_{l,1}, \dots, h_{l,K}$, into its decimal representation $g_l(x)$, i.e.,

$$g_l(x) = \sum_{j=1}^K h_{l,j}(x) \cdot 2^{K-j}. \quad (4)$$

Then, we store the pair $(g_l(x), x)$ as a *key-value* pair into the hash table T_l . Each block $\mathcal{D}_i$ has such a hash table $\{(g_l(x), x)\}_{x \in \mathcal{D}_i}$ where the $(g_l(x), x)$ pairs are sorted in ascending order of the keys. This process is repeated L times to build L hash tables for the block $\mathcal{D}_i$, denoted as $\mathcal{T}_i = \{T_{i,1}, \dots, T_{i,L}\}$. Algorithm 6 describes the procedure of building our LSH framework.

5.1.2 Search in LSH tables. The hash family $\mathcal{H}$ is designed under angular space. It satisfies:

LEMMA 2. Given any two point x_1 and x_2 , let $\theta = \langle x_1, x_2 \rangle$, for a hash function $h \in \mathcal{H}$ defined in Equation 3, the collision probability of $h(x_1)$ and $h(x_2)$ is computed as follows [37, 59].

$$\Pr[h(x) = h(q)] = 1 - \theta/\pi \quad (5)$$

Lemma 2 indicates that the points with small angular distances are more likely to have the same hash value under h . Hence, Upon the LSH tables, we can perform an ANNS query in the angular space for yielding an initial entry point set for each point $x \in \mathcal{D}_i$. To find the ANNS results in $\mathcal{D}_i$ of the point x , we select C candidates v that $g_l(x)$ is close to $g_l(v)$ from L table $T_{i,l}$. As each table is sorted, we can generate the candidates for all points by traversing in $T_{i,l}$ once (Line 9). Therefore, the cost of search in LSH tables is very small. After that, we verify all C candidates of x and choose the best M_0 results as its neighbors. These neighbors serve as the initial set for building the APG, replacing the random initialization in line 2 of Algorithm 1 and line 5 of Algorithm 2, while all other steps remain identical. Since the LSH-based initialization offers a higher quality and more stable starting point compared to random initialization, it significantly mitigates the high computational cost associated with graph construction, leading to faster convergence and improved overall graph quality.

EXAMPLE 2. Figure 3 illustrates the LSH tables in MARIA. Here, $L = 2$ and $K = 2$. Take x_7 as example, $g_1(x_7) = 0$ and $g_2(x_7) = 3$, so x_7 are stored as $(0, x_7)$ in T_1 and $(3, x_7)$ in T_2 . The points in different blocks are stored separately. To generate the initial set for points in $\mathcal{D}_2$, we can simply traverse T_1 and T_2 for $\mathcal{D}_2$ and select the proximity points in the hash table as the candidates to each other. After traversing T_1 , x_7 obtains the candidates x_9 . Similarly, x_7 obtains the candidates x_4 and x_8 in T_2 . By computing the inner product of x_4, x_8 and x_9 to x_7 , we insert x_4 and x_8 into the initial set of x_7 .

5.2 Skip-List-like Mechanism

In Algorithm 2, each point in a block $\mathcal{D}_i$ is connected to $s - i$ other blocks. This would lead to excessive growth in the number of edges, significantly increasing the size of the APG and negatively impacting query efficiency. Moreover, the blocks with the smaller norms are less likely to have the expected AMIPS points than the blocks with larger norms due to the norm bias. Therefore, the fully connected structure improves little over the query quality. To mitigate these issues, we draw inspiration from the skip list

Algorithm 6: LSHInit

Input: Blocks $\{\mathcal{D}_i\}$, Parameters L, K
Output: LSH index $\mathcal{I}_H$

- 1 Generate $L \cdot K$ hash functions $\{h_{l,j}\}$;
- 2 **for each** $\mathcal{D}_i$ **do**
- 3 **for** $x \in \mathcal{D}_i$ **do**
- 4 Compute all L of $g_l(x)$ values based on Eq. 4;
- 5 Insert $(g_l(x), x)$ into hash table T_l ;
- 6 Sort all L tables $\{T_i\}$;
- 7 $\mathcal{T}_i \leftarrow \{T_{i,1}, \dots, T_{i,L}\}$;
- 8 $\mathcal{I}_H \leftarrow \{\mathcal{T}_1, \dots, \mathcal{T}_s\}$;
- 9 Traverse $\mathcal{I}_H$ to generate the initial set for each point;
- 10 **return** $\mathcal{I}_H$

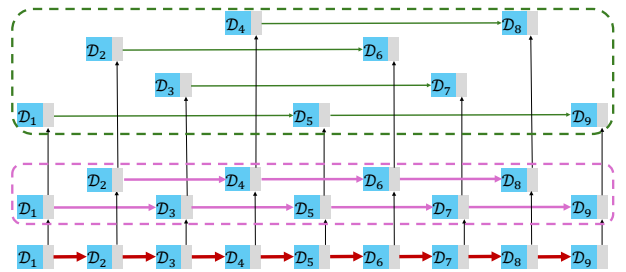


Figure 4: The Illustration of Skip-List-like Mechanism

structure and propose a novel inter-block connection mechanism, which is designed as follows:

- (1) *Skip-list-like connection.* For a point x in $\mathcal{D}_i$, it connects to at most $\lfloor \log(s - 1) \rfloor$ blocks $\mathcal{D}_j$ where $j - i$ is the power of 2.
- (2) *Redistributed edges.* For a point x in $\mathcal{D}_i$, the number of its edges to the block $\mathcal{D}_j$ is $\lceil M'/(j - i) \rceil$.

LEMMA 3. The skip-list-like mechanism reduces the maximum inter-block degree of a point x from $O(M's)$ to $O(M' + \log s)$ and the cost of constructing the inter-block edges from $O(C_Q s)$ to $O(C_Q \log s)$ where C_Q is the cost of search in a block.

The proof of this lemma is provided in Section 6.1. This lemma demonstrates that the skip-list-like mechanism greatly reduces the index size and construction cost. Specifically, the *skip-list-like connection* ensures that the closer are two blocks, the higher is the probability that they are connected, as that in a skip list. Compared to the previous mechanism that the inter-block edges incident from a block $\mathcal{D}_i$ are uniformly distributed in the level of blocks, The *redistributed edges* strategy provides more inter-block edges to the points closer to $\mathcal{D}_i$. Intuitively, if a block $\mathcal{D}_j$ is closer to $\mathcal{D}_i$ than $\mathcal{D}_l$, it is earlier to be accessed than $\mathcal{D}_l$. So, it is more beneficial to generate the high-quality candidates by using a large number of inter-block edges. Moreover, in the previous strategy, the points in block $\mathcal{D}_j$ has $(j - 1) \cdot M'$ inter-block in-edges. Then, the blocks with smaller norm have more in-edges instead, which helps little for the query performance.

EXAMPLE 3. Figure 4 provides an example of our skip-list-like mechanism with $s = 9$. Here, $\mathcal{D}_1$ connects to only 3 blocks and the furthest block connected by it is $\mathcal{D}_5$. Given $M' = 4$, a point x in $\mathcal{D}_1$ has $4 + 2 + 1 = 7$ inter-block neighbors. More than 80% of these neighbors

connect to the points in $\mathcal{D}_2$ and $\mathcal{D}_3$, which are more promising to be the AMIPS neighbor of a query. In the previous strategy, x has $(s-1) \cdot M' = 32$ neighbors, 4 times larger than that in the skip-list-like mechanism. However, a half of them connect to $\mathcal{D}_6, \mathcal{D}_7, \mathcal{D}_8$ and $\mathcal{D}_9$, which are less likely to have the AMIPS results.

5.3 Quantization and Pruning

The query cost of a graph-based method comes from the number of distance computations between q and the neighbors of e_p when accessing e_p . The previous query strategy described in Section 4.3 checks all e_p 's neighbors, which is unnecessary and time-consuming since some neighbors are obviously the unqualified candidates. To address this inefficiency, we introduce a pruning strategy that takes advantage of the properties of LSH functions.

5.3.1 Quantization. Consider that we have $m = L \cdot K$ hash functions, we concatenate all m hash values $h_{l,j}(x)$ for the point x as its quantization $Q(x)$. Then, we can estimate $\theta = \langle q, x \rangle$ for two points by using their quantization values.

LEMMA 4. For any two points q and x , denote $\theta = \langle q, x \rangle$ and

$$\beta(q, x) = \|Q(q), Q(x)\|_H = \sum_{j=1}^m \mathbb{I}(Q_j(q) \neq Q_j(x)) \quad (6)$$

where $\|\cdot, \cdot\|_H$ is the Hamming distance, then $\theta' = \frac{\pi\beta(q, x)}{m}$ is an unbiased estimator of θ .

Lemma 4 indicates that we can estimate θ accurately by using θ' when m is large enough as θ' is an unbiased estimator of θ . Since $q^\top x = \|q\| \cdot \|x\| \cdot \cos \theta$, we can further estimate $q^\top x$ before computing it. $\beta(q, x)$ can be quickly computed by the bitwise XOR operation. So, we can estimate $q^\top x$ efficiently and accurately.

5.3.2 Pruning Condition. Given a tolerant error $0 < \delta < 1$, the pruning condition based on the quantization values is as follows:

$$I_{\min} > I_{ub} = \|q\| \cdot \|x\| \cdot \cos(\theta' - z\sqrt{\theta'(\pi - \theta')/m}), \quad (7)$$

where $I_{\min}$ is the minimal inner product between q and the points in the result set, I_{ub} is the current found k -th maximum inner product and z is the quantile of the standard normal distribution at $1 - \delta/2$ to control the estimation error and efficiency. The intuition of the pruning condition is that when $I_{\min}$ is greater than I_{ub} , $I_{\min}$ will also be greater than $q^\top x$ with high probability $1 - \delta$. Then, it is reasonable to discard x without computing $q^\top x$. The correctness of this pruning condition is proven in Section 6. With a bit of estimation cost, this pruning condition significantly reduces the computation cost and still guarantees the query quality.

EXAMPLE 4. As shown in Figure 3, $m = L \cdot K = 4$. Take x_4 as example, $Q(x_8) = [g_1(x_8); g_2(x_8)] = [2; 3] = \overline{1011}$ and $Q(q_1) = [0; 3] = \overline{0011}$. So, $\beta(q_1, x_8) = 1$ and we can estimate $\theta = \langle q_2, x_4 \rangle$ as $\theta' = \pi\beta(q, x)/m = \pi/4$. Assume that $I_{\min} = q_1^\top x_{11} = 139$, $x_8 = (6, 0)$ and $q_1 = (11, 8)$, let z be $1/\sqrt{3}$, then I_{ub} can be estimated as 75.4. Hence, we can prune x_8 without computing $q^\top x_8$.

6 THEORETICAL ANALYSIS

We first provide the proofs of lemmas in Sections 4 and 5. Then, we analyze the space and time complexity of our algorithm.

Algorithm 7: MARIA Search

Input: A query q , the APG I_G and parameters k and ef
Output: k points in I_G
 /* Replace line 1 in Algorithm 4 */
 1 $EP \leftarrow M_0$ points identified by I_H on $\mathcal{D}_1$;
 /* Add after line 10 in Algorithm 3 */
 2 **if** $I_{\min} > I_{ub}$ **then**
 /* Same as Line 8 - 12 in Algorithm 3 */

6.1 Proofs

We prove Lemma 1 that reveals the equivalence between the AMIPS and ANNS in a block.

PROOF OF LEMMA 1. Denote x^* as the exact angular neighbor of q in the block $\mathcal{D}_i$ and y^* as the exact MIPS neighbor of q in the block $\mathcal{D}_i$, $\theta_1 = \langle q, x^* \rangle$ and $\theta_2 = \langle q, y^* \rangle$, we have $\theta_1 < \theta_2$ and $\cos \theta_1 \geq \cos \theta_2$. Given the definition of the norm partition, we have $\|y^*\| \leq x^*/\alpha$. Hence, $q^\top y^* = \|q\| \cdot \|y^*\| \cdot \cos \theta_2 \leq \|q\| \cdot x^*/\alpha \cdot \cos \theta_1 = q^\top x^*/\alpha$. So, x^* is a correct α -AMIPS neighbor of q . $\square$

Then, we prove the Lemma 3, which demonstrates the advantages of the skip-list-like mechanism in terms of both index size and construction cost.

PROOF OF LEMMA 3. In the naive inter-connection strategy, each point in a block $\mathcal{D}_i$ will connect to other $(s-i)$ blocks and requires $(s-i)$ searches. So, the average inter-block degree is $s/2 \cdot M' = O(sM')$ and the average search cost is $O(C_Q s)$. However, in the skip-list-like mechanism, each point in a block $\mathcal{D}_i$ will connect to other $O(\log s)$ blocks, so the average search cost is $O(C_Q \log s)$. The maximum inter-block degree is

$$\sum_{l=1}^{\lceil \log M' \rceil} \frac{M}{2^{l-1}} + \lceil \log(s-1) \rceil - \lceil \log M' \rceil < 2M' + \lceil \log s \rceil,$$

which is $O(M' + \log s)$. $\square$

Next, we analyze the property of $\beta(q, x)$ and prove Lemma 4 that reveals the effectiveness of the quantization. Since $\Pr[h_j(q) \neq h_j(x)] = \theta/\pi$ and all of m hash functions are independent, $\beta(q, x)$ follows $B(m, \theta/\pi)$ based on the definition of the binomial distribution [16, 17, 54].

PROOF OF LEMMA 4. As $\beta(q, x) \sim B(m, \theta/\pi)$, $\mathbb{E}[\beta(q, x)] = m \cdot \theta/\pi$. Therefore, $\mathbb{E}[\theta'] = \frac{\pi}{m} \mathbb{E}[\beta(q, x)] = \theta$, which indicates θ' is an unbiased estimation of θ . $\square$

Based on the Lemmas 4, we have the following conclusion that reveals the correctness of the pruning condition:

LEMMA 5. Given a tolerant error $\delta \in (0, 1)$, z is the quantile of the standard normal distribution at $1 - \delta/2$. For two points q and x , denote θ' as the estimator of their angle under the m -length quantization, $\theta = \langle q, x \rangle$ falls into the confidential interval $[\theta' - z\sqrt{\theta'(\pi - \theta')/m}, \theta' + z\sqrt{\theta'(\pi - \theta')/m}]$ with the probability $1 - \delta$.

PROOF. As $\beta(q, x) \sim B(m, \theta/\pi)$, for the success probability $p = \theta/\pi$, the $1 - \delta$ confidence interval of p can be $[p' - z\sqrt{\frac{p'(1-p')}{m}}, p' +$

$z\sqrt{\frac{p'(1-p')}{m}}$ based on the Wald method [1], where $p' = \beta(q, x)/\pi$ is the proportions of successes. By rewriting it, the confidential interval of θ can be $[\theta' - z\sqrt{\theta'(\pi - \theta')/m}, \theta' + z\sqrt{\theta'(\pi - \theta')/m}]$. $\square$

Based on this lemma, $\theta \geq \theta' - z\sqrt{\theta'(\pi - \theta')/m}$ holds with the probability $1 - \delta$. When it holds, $I_{ub} = \|q\| \cdot \|x\| \cdot \cos(\theta' - z\sqrt{\theta'(\pi - \theta')/m}) \geq q^\top x = \|q\| \cdot \|x\| \cdot \cos \theta$. So, when I_{ub} is less than $I_{\min}$, it is proper to discard x .

6.2 Time and Space Complexity

To analyze the time and space complexity, we first discuss the setting of the related parameters in MARIA, including the maximum inner-block degree M , maximum inter-block degree between two blocks M' , the number of blocks s , the number of hash tables L and the number of hash functions used in a table K . As that in mainstream graph based methods [12, 36, 46, 46, 62], we fix M and M' as a constant. s depends on the range of norms in $\mathcal{D}$ and is affected little by n . Hence, $s = O(1)$. In addition, L and K can also be set as constants for a fast retrieval [24, 33].

The construction and query cost of MARIA depends on the cost of search in a block, C_Q . To construct APG, we conduct at most $O(\log s)$ searches in the APG for a point in $\mathcal{D}$. So the construction cost is $O(nC_Q \log s)$. Similarly, the worst query cost is $O(sC_Q)$ when all blocks are checked. The structure of an APG is so complicated that it becomes very difficult for analyzing C_Q [40, 53]. Some works [34, 52] regard C_Q as $O(\log n)$ by considering the “six degrees of separation” assumption [7, 56]. In addition, C_Q is proven to be $O(1)$ if the entry point is close enough to the query [58]. Based on these conclusions, C_Q is very likely to be bounded by $O(\log n)$. Consider that $s = O(1)$, the construction cost of MARIA is $O(n \log n)$ and the query cost in MARIA is $O(\log n)$. Then, each point has at most $O(M)$ inner-block edges and $O(M' + \log s)$ inter-block edges, so the index size is $O(n(M + M' + \log s)) = O(n)$.

7 EXPERIMENT

In this section, we conduct extensive experiments on real-world datasets to provide a comprehension analysis of MARIA. We implement MARIA² and the competitors in C++ compiled with g++9.5.0 using -Ofast optimization and openMP for parallelism. All experiments are run on Ubuntu 22.04.5 LTS with 4 Intel(R) Xeon(R) Gold 6218 CPUs (160 threads) and 1.5 TB RAM.

7.1 Experimental Settings

Datasets. We employ 6 real-world datasets with varied cardinality of from 0.6M to 1B, whose information is summarized in Table 2 in the ascending order of their sizes. These datasets are widely used in ANNS and AMIPS [32, 36, 59]. Especially, the two billion-scale datasets, Deep1B and Yandex dataset³ are used for evaluating the scalability. For each dataset, we randomly select 10K query points from its query set to study the query performance.

Competitors. To demonstrate the indexing and query performance of MARIA, we compare it with the following in-memory AMIPS methods, including LSH-based and graph-based methods since they

Table 2: Summary of Datasets

Datasets	Card.	Dim.	Size (GB)
YahooMusic	625,000	300	0.72
Deep1M	1,000,000	256	1.00
Tiny5M	5,000,000	384	4.77
Tiny80M	79,302,017	384	113
Deep1B	1,000,000,000	96	358
Yandex	1,000,000,000	200	745

have been proven to be superior than other kinds of methods [52, 58]. All methods are run with a total of 160 threads in memory.

- **FARGO**⁴ [59] divides the dataset via norm partition where each block has at most N_0 points. Then, it adopts L K -dimensional hash tables to answer c -AMIPS problems in each block.
- **ip-NSW**⁵ [36] adopts the classic APG for ANNS, HNSW [32] to solve AMIPS problem by replacing the similarity function in HNSW from L_2 distance as inner product.
- **ip-NSW**⁺⁶ [32] improves the query performance of ip-NSW by building another APG in the angular space to generate the better entry points for search in ip-NSW.
- **NAPG**⁷ [46] improves the graph quality of ip-NSW by adjusting the edge selection strategy in ip-NSW for remaining more neighbors for each vertex.

Parameter Settings. For the value of k in k AMIPS queries, as most AMIPS algorithms [36, 46, 59] vary k from 1 to 100 and k will not affect the performances significantly [58], we consider $k = 50$ for all algorithms in the default settings. Following the settings in the paper or source code of our competitors, we adopt the fixed parameters for each algorithm. For MARIA, we set $\alpha = 0.85$, $L = 4$, $K = 16$, $M = 24$, $efC = 80$ and $M' = 16$. For ip-NSW, ip-NSW⁺ and NAPG, as they are all improved on top of HNSW, we set $M = 24$ and $efC = 80$ for them. For FARGO, we set $N_0 = 4096$, $c = 0.8$, $L = 5$ and $K = 12$ as suggested in the paper.

Evaluation Metric. We compare the algorithms in terms of index size, indexing time, QPS (queries per second) and average query accuracy. QPS indicates the number of queries answered by an algorithm within a second. We adopt recall to measure the query accuracy. Given a query point q , for a k AMIPS query, assume that the algorithm return the set $R = \{x_1, \dots, x_k\}$ and the exact k MIPS of q is $R^* = \{x_1^*, \dots, x_k^*\}$, then recall are defined as follows [59],

$$Recall = \frac{|R \cap R^*|}{k}. \quad (8)$$

7.2 Self Evaluation

In this subsection, we demonstrate the effectiveness of our three optimizations: the LSH framework, the skip-list-like mechanism, and the quantization-based pruning condition, on both the indexing and query performance of MARIA. We compare MARIA with the following variations: (i) **MARIA-noL**, which omits the LSH index; (ii) **MARIA-RC**, where points in a block $\mathcal{D}_i$ connect to at most $O(\log s - i)$ other blocks $\mathcal{D}_j$ for $j > i$; and (iii) **MARIA-noQ**, which excludes quantizations and pruning conditions. Due to space constraints, we

²<https://github.com/Jacyhust/MARIA>

³<https://storage.yandexcloud.net/yandex-research/ann-datasets/T2L/base.1B.fb1n>

⁴<https://github.com/Jacyhust/FARGO>

⁵<https://github.com/stanis-morozov/ip-nsw>

⁶<https://github.com/jerry-liujie/ip-nsw/tree/GraphMIPS>

⁷https://github.com/ThrunGroup/BanditMIPS/tree/camera_ready/algorithms/napg

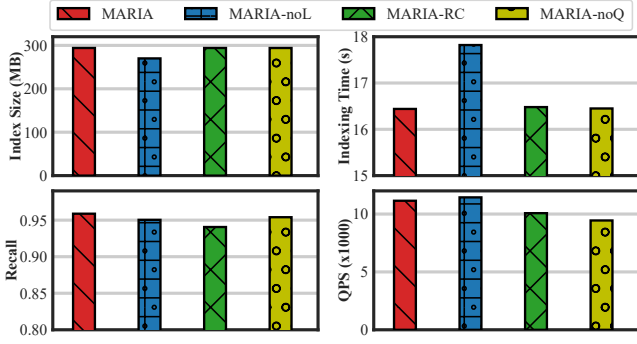


Figure 5: Self Evaluation

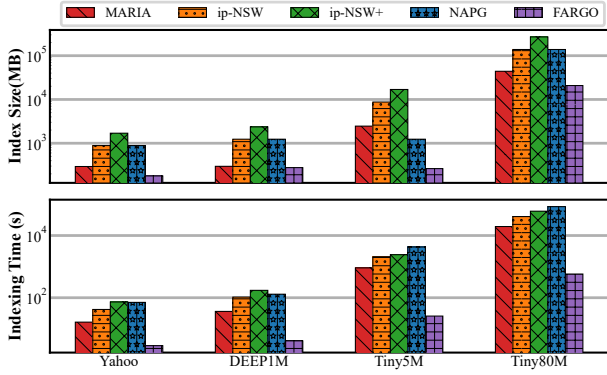


Figure 6: Indexing Performance

report results only on the YahooMusic dataset for multiple times, as shown in Figure 5.

7.2.1 Evaluation of the LSH Framework. We compare MARIA with MARIA-noL to demonstrate the effectiveness of the LSH framework. Regarding indexing performance, MARIA has a 10% larger index size due to the LSH framework’s space consumption but spends 25% less time constructing the index. This indicates that the higher-quality entry points generated by the LSH framework reduce the search cost in the APG. For query performance, MARIA achieves higher recall with slightly lower QPS compared to MARIA-noL, suggesting that MARIA-noL tends to terminate the search at suboptimal local maxima due to random entry points.

7.2.2 Evaluation of the Skip-List-like Mechanism. We compare MARIA with MARIA-RC to evaluate the skip-list-like mechanism. For indexing performance, MARIA-RC results in a similar index size and indexing time as MARIA, due to the same number of inter-block edges. For query performance, MARIA-RC achieves lower recall and QPS, indicating that the skip-list-like mechanism generates higher-quality inter-block edges.

7.2.3 Evaluation of the Quantization-Based Pruning. We compare MARIA with MARIA-noQ to assess the quantization-based pruning. For indexing performance, MARIA and MARIA-noQ exhibit similar index sizes and indexing times. However, regarding query performance, MARIA achieves higher QPS than MARIA-noQ while maintaining nearly the same recall, indicating that the pruning condition effectively filters out many unqualified points.

7.3 Evaluation of Indexing Performance

In this set of experiments, we evaluated the index size and indexing time of all algorithms. Our findings are illustrated in Figure 6. We made the following observations when comparing all algorithms based on these two metrics. (1) Among the graph-based algorithms, MARIA has the smallest index size, surpassed only by the LSH-based method, FARGO. FARGO achieves the smallest index size by storing only $L = 5$ hash tables. Both ip-NSW and NAPG have similar index sizes due to their identical maximum degree, which determines the space allocated for the APG. Ip-NSW⁺ has the largest index size as it requires an additional APG to generate entry points. (2) Among the graph-based algorithms, MARIA has the smallest indexing time, approximately 50% of the second-best method, ip-NSW, highlighting the effectiveness of the LSH framework and quantization-based pruning condition in enhancing indexing efficiency. Ip-NSW⁺ or NAPG exhibits the largest indexing time on different datasets. The high construction cost of ip-NSW⁺ stems from the additional cost of building another APG. NAPG’s construction efficiency varies with datasets as it adjusts edge selection rules based on data distribution. Additionally, the high construction cost of NAPG on Tiny5M and Tiny80M highlights the inefficiency of its proposed strategy.

7.4 Evaluation of Query Performance

In these experiments, we analyze the relationship between recall and QPS using the Recall-QPS curve to evaluate the query performance of all algorithms by varying the default number of points to be checked. The results for four datasets are shown in Figure 7. NAPG exhibits significantly poorer performance on Tiny5M and Tiny80M, so its results are not reported for these datasets. From the figures, we observe the following: (1) As QPS decreases, all algorithms achieve higher recall, consistent with the trade-off between recall and QPS. Among all algorithms, MARIA achieves the highest QPS for the same recall, indicating the best balance between query quality and efficiency. (2) FARGO achieves the second-best query performance on YahooMusic, Tiny5M, and Tiny80M but surprisingly exhibits the lowest performance on DEEP1M. This difference arises from the varied norm distributions of these datasets. In YahooMusic, Tiny5M, and Tiny80M, the wide range of point norms allows finding satisfactory AMIPS results in a few blocks after norm partitioning. However, in DEEP1M, points have nearly identical norms. The difference in FARGO’s query performance highlights the effectiveness of norm partitioning in the AMIPS problem. (3) The recall of ip-NSW and ip-NSW⁺ nearly converges at 0.95 on Tiny5M and 0.85 on Tiny80M, even when increasing the candidate size ef to boost query accuracy, indicating a significant norm bias issue. Combined with the analysis on FARGO, this underscores the critical importance of norm partitioning in the AMIPS problem.

7.5 Scalability Test

In this set of experiments, we evaluate the scalability of all algorithms on two billion-scale datasets, DEEP1B and Yandex. We assess indexing performance and query performance by varying data cardinality in the range of $\{0.1M, 1M, 10M, 100M, 1B\}$. Additionally, we examine the extra index size and indexing time from the LSH framework in MARIA (denoted as LSH-In-MARIA). ip-NSW, ip-NSW⁺ and NAPG fails to build indexes on Yandex due to

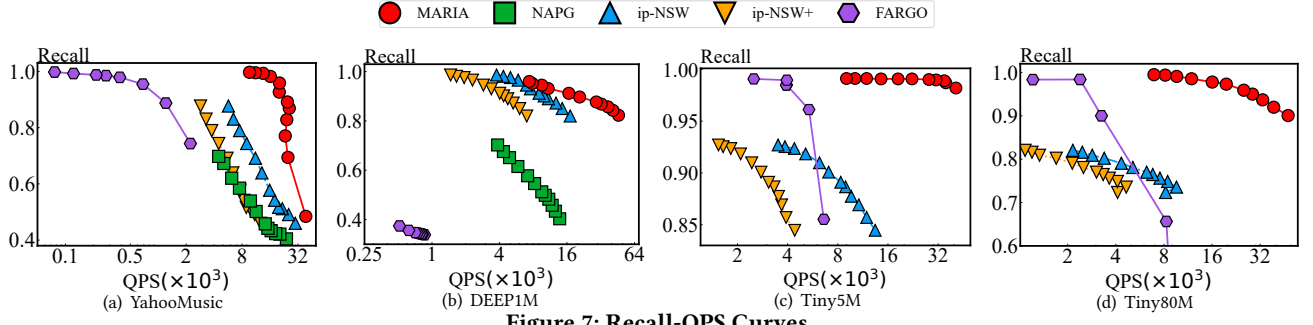


Figure 7: Recall-QPS Curves

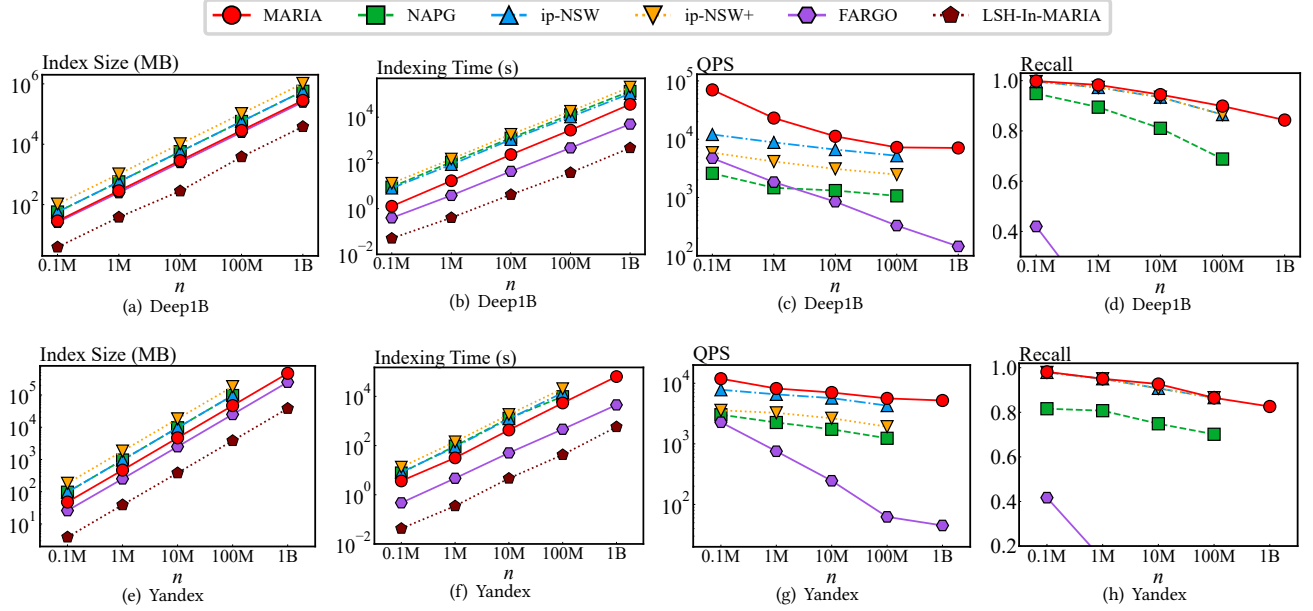


Figure 8: Scalability Test

out of memory, so we do not report their performance on Yandex. All results are shown in Figure 8. From the figure, we observe the following: (1) MARIA achieves the smallest index size and indexing time among all graph-based methods, indicating strong support for large-scale datasets. Moreover, the indexing time and index size of LSH-In-MARIA are less than 1% of those in MARIA, demonstrating its lightweight and efficiency. Additionally, as an LSH index, LSH-In-MARIA shows superior indexing performance compared to FARGO. (2) Comparing index sizes across different datasets, we found that index size is nearly linear with dataset cardinality n in all algorithms. This is because these algorithms have fixed degrees or a fixed number of hash tables. The indexing time also follows this trend, indicating that the cost of finding neighbors for a point changes slightly with data cardinality n . (3) MARIA consistently achieves the highest QPS and recall among all algorithms on these two datasets. Moreover, MARIA’s QPS decreases very gradually with n . Notably, when n changes from 100M to 1B, MARIA’s QPS remains nearly unchanged, further demonstrating its superior scalability in query performance. (4) Although FARGO achieves a much smaller index size and indexing time than all graph-based methods, its query performance is poor, especially for datasets with large

cardinality. As n increases, FARGO’s QPS decreases rapidly, becoming uncompetitive with graph-based methods. FARGO’s inferior QPS is due to its $O(n)$ theoretical query cost. The query accuracy of FARGO also dramatically decreases with n , highlighting the limited query performance of LSH-based methods on large-scale datasets.

8 CONCLUSIONS

In this paper, we introduce a novel framework called MARIA for AMIPS queries in high-dimensional spaces on large-scale datasets. MARIA addresses challenges such as the norm bias issue and the non-metric nature of the inner product by using a norm partition-based graph structure. We also implement three optimizations to enhance query performance and scalability. Extensive experiments show that MARIA achieves a 2-4 $\times$ speedup in indexing and query costs compared to leading competitors, while providing superior query quality, making it suitable for billion-scale datasets. In the future, we will study the disk based version and maintenance of MARIA for better supporting its utilization in vector databases.

REFERENCES

- [1] Alan Agresti and Brent A Coull. 1998. Approximate is better than “exact” for interval estimation of binomial proportions. *The American Statistician* 52, 2 (1998), 119–126.
- [2] Thomas Dybdahl Ahle, Rasmus Pagh, Ilya P. Razenshteyn, and Francesco Silvestri. 2016. On the Complexity of Inner Product Similarity Join. In *PODS*. 151–164.
- [3] Alexandr Andoni and Piotr Indyk. 2016. *LSH Algorithm and Implementation (E2LSH)*. <https://www.mit.edu/~andoni/LSH>
- [4] Alexandr Andoni, Piotr Indyk, Huy L. Nguyen, and Ilya P. Razenshteyn. 2014. Beyond Locality-Sensitive Hashing. In *SODA*. 1018–1028.
- [5] Azure. 2024. Available at: <https://azure.microsoft.com/>
- [6] Yoram Bachrach, Yehuda Finkelstein, Ran Gilad-Bachrach, Liran Katzir, Noam Koenigstein, Nir Nave, and Ulrich Paquet. 2014. Speeding up the Xbox recommender system using a Euclidean transformation for inner-product spaces. In *RecSys*. 257–264.
- [7] Lars Backstrom, Paolo Boldi, Marco Rosa, Johan Ugander, and Sebastiano Vigna. 2012. Four degrees of separation. In *Proceedings of the 4th annual ACM Web science conference*. 33–42.
- [8] Robert M. Bell and Yehuda Koren. 2007. Lessons from the Netflix prize challenge. *SIGKDD Explorations* 9, 2 (2007), 75–79.
- [9] Moses Charikar. 2002. Similarity estimation techniques from rounding algorithms. In *STOC*. 380–388.
- [10] Hongming Chen, Ola Engkvist, Yinhai Wang, Marcus Olivecrona, and Thomas Blaschke. 2018. The rise of deep learning in drug discovery. *Drug discovery today* 23, 6 (2018), 1241–1250.
- [11] Lijie Chen. 2018. On The Hardness of Approximate and Exact (Bichromatic) Maximum Inner Product. *Electronic Colloquium on Computational Complexity (ECCC)* 25 (2018), 26.
- [12] Meng Chen, Kai Zhang, Zhenying He, Yinan Jing, and X. Sean Wang. 2024. RoarGraph: A Projected Bipartite Graph for Efficient Cross-Modal Approximate Nearest Neighbor Search. *Proc. VLDB Endow.* 17, 11 (2024), 2735–2749.
- [13] ACM SIGMOD 2023 Programming Contest. 2023. Available at: https://people.cs.rutgers.edu/~dd903/sigmodpc2023/leader2023_final.html
- [14] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM conference on recommender systems*. 191–198.
- [15] Xinyan Dai, Xiao Yan, Kelvin Kai Wing Ng, Jiu Liu, and James Cheng. 2020. Norm-Explicit Quantization: Improving Vector Quantization for Maximum Inner Product Search. In *AAAI*. 51–58.
- [16] Binomial distribution. 2024. Available at: https://en.wikipedia.org/wiki/Binomial_distribution
- [17] Bernoulli distribution. 2024. Available at: https://en.wikipedia.org/wiki/Bernoulli_distribution
- [18] Wei Dong, Moses Charikar, and Kai Li. 2011. Efficient k-nearest neighbor graph construction for generic similarity measures. In *WWW*. 577–586.
- [19] Marco Fraccaro, Ulrich Paquet, and Ole Winther. 2016. Indexable Probabilistic Matrix Factorization for Maximum Inner Product Search. In *AAAI*. 1554–1560.
- [20] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast Approximate Nearest Neighbor Search With The Navigating Spreading-out Graph. *PVLDB* 12, 5 (2019), 461–474.
- [21] Junhao Gan, Jianlin Feng, Qiong Fang, and Wilfred Ng. 2012. Locality-sensitive hashing scheme based on dynamic collision counting. In *SIGMOD*. 541–552.
- [22] Aristides Gionis, Piotr Indyk, and Rajeev Motwani. 1999. Similarity Search in High Dimensions via Hashing. In *VLDB*. 518–529.
- [23] Ben Harwood and Tom Drummond. 2016. FANNG: Fast Approximate Nearest Neighbour Graphs. In *CVPR*. IEEE Computer Society, 5713–5722.
- [24] Qiang Huang, Guihong Ma, Jianlin Feng, Qiong Fang, and Anthony K. H. Tung. 2018. Accurate and Fast Asymmetric Locality-Sensitive Hashing Scheme for Maximum Inner Product Search. In *KDD*. 1561–1570.
- [25] Alexis Joly and Olivier Buisson. 2011. Random maximum margin hashing. In *CVPR*. IEEE Computer Society, 873–880.
- [26] Omid Keivani, Kaushik Sinha, and Parikshit Ram. 2018. Improved maximum inner product search with better theoretical guarantee using randomized partition trees. *Machine Learning* 107, 6 (2018), 1069–1094.
- [27] Rolf Klein. 2016. Voronoi Diagrams and Delaunay Triangulations. In *Encyclopedia of Algorithms*. 2340–2344.
- [28] Noam Koenigstein, Parikshit Ram, and Yuval Shavitt. 2012. Efficient retrieval of recommendations in a matrix factorization framework. In *CIKM*. 535–544.
- [29] Naama Kraus, David Carmel, Idit Keidar, and Meni Orenbach. 2016. NearBucket-LSH: Efficient Similarity Search in P2P Networks. In *SISAP (Lecture Notes in Computer Science)*, Vol. 9939. 236–249.
- [30] Quoc Le and Tomas Mikolov. 2014. Distributed representations of sentences and documents. In *International conference on machine learning*. PMLR, 1188–1196.
- [31] Jinfeng Li, Xiao Yan, Jian Zhang, An Xu, James Cheng, Jie Liu, Kelvin K. W. Ng, and Ti-chung Cheng. 2018. A General and Efficient Querying Method for Learning to Hash. In *SIGMOD*. 1333–1347.
- [32] Jie Liu, Xiao Yan, Xinyan Dai, Zhirong Li, James Cheng, and Ming-Chang Yang. 2020. Understanding and Improving Proximity Graph Based Maximum Inner Product Search. In *AAAI*. 139–146.
- [33] Kejing Lu, Hongya Wang, Wei Wang, and Mineichi Kudo. 2020. VHP: Approximate Nearest Neighbor Search via Virtual Hypersphere Partitioning. *Proc. VLDB Endow.* 13, 9 (2020), 1443–1455.
- [34] Yury A. Malkov and Dmitry A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* 42, 4 (2020), 824–836.
- [35] Milvus. 2023. Available at: <https://milvus.io/>
- [36] Stanislav Morozov and Artem Babenko. 2018. Non-metric Similarity Graphs for Maximum Inner Product Search. In *NeurIPS*. 4726–4735.
- [37] Behnam Neyshabur and Nathan Srebro. 2015. On Symmetric and Asymmetric LSHs for Inner Product Search. In *ICML*, Vol. 37. 1926–1934.
- [38] Naoki Ono and Yusuke Matsui. 2023. Relative NN-Descent: A Fast Index Construction for Graph-Based Approximate Nearest Neighbor Search. In *ACM Multimedia*. ACM, 1659–1667.
- [39] Pinecone. 2024. Available at: <https://www.pinecone.io/>
- [40] Liudmila Prokhorenkova and Aleksandr Shekhovtsov. 2020. Graph-based Nearest Neighbor Search: From Practice to Theory. In *ICML (Proceedings of Machine Learning Research)*, Vol. 119. PMLR, 7803–7813.
- [41] Parikshit Ram and Alexander G. Gray. 2012. Maximum inner-product search using cone trees. In *KDD*. 931–939.
- [42] Fumin Shen, Wei Liu, Shaoting Zhang, Yang Yang, and Heng Tao Shen. 2015. Learning Binary Codes for Maximum Inner Product Search. In *ICCV*. 4148–4156.
- [43] Anshumali Shrivastava. 2015. Improved Asymmetric Locality Sensitive Hashing (ALSH) for Maximum Inner Product Search (MIPS). In *UAI*. AUAI Press, 812–821.
- [44] Anshumali Shrivastava and Ping Li. 2014. Asymmetric LSH (ALSH) for Sublinear Time Maximum Inner Product Search (MIPS). In *NIPS*. 2321–2329.
- [45] Suhas Jayaram Subramanya, Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnaswamy, and Rohan Kadekodi. 2019. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. In *NeurIPS*. 13748–13758.
- [46] Shulong Tan, Zhaozhuo Xu, Weijie Zhao, Hongliang Fei, Zhixin Zhou, and Ping Li. 2021. Norm Adjusted Proximity Graph for Fast Inner Product Retrieval. In *KDD*. ACM, 1552–1560.
- [47] Shulong Tan, Zhixin Zhou, Zhaozhuo Xu, and Ping Li. 2019. On Efficient Retrieval of Top Similarity Vectors. In *EMNLP/TJCNLP (1)*. Association for Computational Linguistics, 5235–5245.
- [48] Yufei Tao, Ke Yi, Cheng Sheng, and Panos Kalnis. 2009. Quality and efficiency in high dimensional nearest neighbor search. In *SIGMOD*. 563–576.
- [49] Yao Tian, Xi Zhao, and Xiaofang Zhou. 2022. DB-LSH: Locality-Sensitive Hashing with Query-based Dynamic Bucketing. In *ICDE*. 2250–2262.
- [50] Mo Tiwari, Ryan Kang, Jaeyong Lee, Donghyun Lee, Christopher Piech, Sebastian Thrun, Ilan Shomorony, and Martin Jinze Zhang. 2024. Faster Maximum Inner Product Search in High Dimensions. In *ICML*. OpenReview.net.
- [51] Vamana vs. HNSW Exploring ANN algorithms Part 1. 2022. Available at: <https://weaviate.io/blog/ann-algorithms-vamana-vs-hnsw>
- [52] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A Comprehensive Survey and Experimental Comparison of Graph-Based Approximate Nearest Neighbor Search. *PVLDB* 14, 11 (2021), 1964–1978.
- [53] Zeyu Wang, Qitong Wang, Xiaoxing Cheng, Peng Wang, Themis Palpanas, and Wei Wang. 2024. Steiner-Hardness: A Query Hardness Measure for Graph-Based ANN Indexes. *CoRR* abs/2408.13899 (2024).
- [54] Eric W Weisstein. 2002. Bernoulli distribution. <https://mathworld.wolfram.com/> (2002).
- [55] Xiao Yan, Jinfeng Li, Xinyan Dai, Hongzhi Chen, and James Cheng. 2018. Norm-Ranging LSH for Maximum Inner Product Search. In *NeurIPS*. 2956–2965.
- [56] Lei Zhang and Wanqing Tu. 2009. Six degrees of separation in online society. (2009).
- [57] Xi Zhao, Zhonghan Chen, Kai Huang, Ruiyuan Zhang, Bolong Zheng, and Xiaofang Zhou. 2024. Efficient Approximate Maximum Inner Product Search Over Sparse Vectors. In *ICDE*. IEEE, 3961–3974.
- [58] Xi Zhao, Yao Tian, Kai Huang, Bolong Zheng, and Xiaofang Zhou. 2023. Towards Efficient Index Construction and Approximate Nearest Neighbor Search in High-Dimensional Spaces. *Proc. VLDB Endow.* 16, 8 (2023), 1979–1991.
- [59] Xi Zhao, Bolong Zheng, Xiaomeng Yi, Xiaofan Luan, Charles Xie, Xiaofang Zhou, and Christian S. Jensen. 2023. FARGO: Fast Maximum Inner Product Search via Global Multi-Probing. *Proc. VLDB Endow.* 16, 5 (2023), 1100–1112.
- [60] Bolong Zheng, Xi Zhao, Lianggui Weng, Nguyen Quoc Viet Hung, Hang Liu, and Christian S. Jensen. 2020. PM-LSH: A Fast and Accurate LSH Framework for High-Dimensional Approximate NN Search. *Proc. VLDB Endow.* 13, 5 (2020), 643–655.
- [61] Yuxin Zheng, Qi Guo, Anthony K. H. Tung, and Sai Wu. 2016. LazyLSH: Approximate Nearest Neighbor Search for Multiple Distance Functions with a Single Index. In *SIGMOD*. 2023–2037.
- [62] Zhixin Zhou, Shulong Tan, Zhaozhuo Xu, and Ping Li. 2019. Möbius Transformation for Fast Inner Product Search on Graph. In *NeurIPS*. 8216–8227.